

10

Introduction to Python Modules

10.1 INTRODUCTION

As our programs become bigger and more complex, the need to organize our code becomes greater. A large and complex program should be divided into files/functions that perform specific tasks. As we write more and more functions in a program, we should consider organizing them by storing them in modules.

A module is simply a file that contains a Python code. When we break a program into modules, each module should contain functions that perform related tasks. *For example*, suppose we are writing an accounting system. We would store all the account receivable functions in their own module and the account payable functions and all the payroll functions in their respective modules. This approach, called modularization, makes a program easier to understand, test and maintain.

Even tiny real-world applications contain thousands of lines of code. In fact, applications that contain millions of lines of code are somewhat common. Imagine trying to work with a file large enough to contain millions of lines of code—you would never find anything so easily. In short, we need some method to organize the code into smaller pieces that are easy to manage, much like the examples in this book. The Python solution is to place the code in separate code groupings called **modules**. While a function is a grouping of instructions, a module is a grouping of functions. As we know that when a program grows, function is used to simplify the code and to avoid repetition. For a complex problem, it may not be feasible to manage the code in one single file. Then, the program is divided into different parts under different levels, called modules. Also, suppose we have created some functions in a program and we want to reuse them in another program. In that case, we can save those functions under a module and reuse them. A module is created as a python (.py) file containing a collection of function definitions.

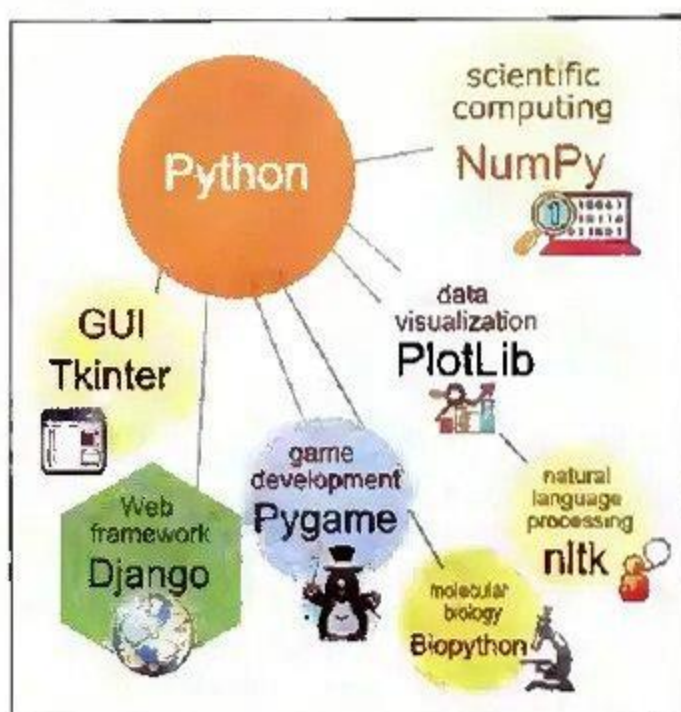


Fig. 10.1: Python Libraries

Commonly-used modules that contain source code for generic needs are called **libraries**.

Therefore, when we speak about working with libraries in Python, we are, in fact, working with modules that are created inside the package(s) or, say, library. Thus, a Python program comprises three main components:

- Library or Package
- Module
- Functions/Sub-modules

Relation between a module, package and library in Python:

A **module** is a file containing Python definitions, functions, variables, classes and statements with `.py` extension. On the other hand, a **Python package** is simply a directory of Python module(s).

A **library** in Python is a collection of various packages. Conceptually, there is no difference between a package and Python library. In Python, a library is used to loosely describe a collection of core or main modules.

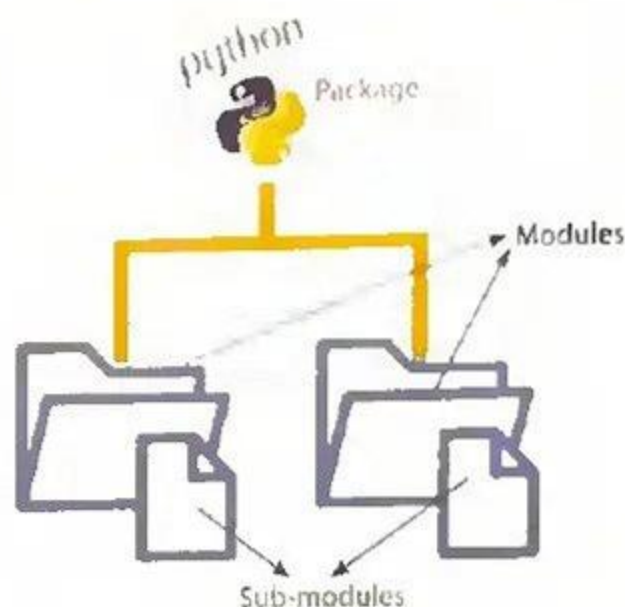


Fig. 10.2: Relation between a Package and Modules

10.2 MODULES IN PYTHON

Modules are used to categorize Python code into smaller parts. Act of partitioning a program into individual components(modules) is called modularity. A module is a separate unit in itself. In other words, a module is simply a Python file where statements, classes, objects, functions, constants and variables are defined. Grouping similar codes into a single file makes it easy to access. *For example*, if the content of a book is not indexed or categorized into individual chapters, the book will become boring and complicated. Hence, dividing the book into chapters makes it easy to understand. In the same way, Python modules are files which have a similar code. Thus, a module simplifies a Python code where classes, variables and functions are defined.

CTM: A module is a file consisting of a Python code that can define functions, classes and variables related to a particular task. A module allows us to organize our code by grouping related codes, which makes the code easier to understand and use.

Python module may contain the following objects:

Objects	Description
docstring	String literals enclosed with triple quotes that appear right after the definition of a function, class, method or module. It is mainly used for documentation and can be accessed using: <code>(__doc__)</code> attribute.
Variables and constants	For storing values
Classes	To create blueprint of any object
Objects	Object is an instance of class. It represents class in real world.
Statements	Instruction
Functions	Group of statements to perform a specific task.

Advantages of Python Modules

Python modules have the following advantages:

1. Putting code into module is useful because of its ability to import the module functionality.
2. A module can be used in some other Python code. Hence, it provides the facility of code reusability.
3. It creates numbers of well-defined, documented boundaries within a program.
4. A module allows us to logically organize our Python code.
5. Grouping a related code into a module makes the code easier to understand and use.
6. Similar types of attributes can be categorized and placed in a single module.

Module Names

A module name is the file name with the .py extension. When we have a file named empty.py, empty is the module name. The `__name__` is a variable that holds the name of the module being referenced.

The current module or the module being executed (also called the main module) has a special name: `'__main__'`. With this name, it can be referenced from the Python code.

A module's file name should end with .py. If the module's file name does not end with .py as the extension, we will not be able to import it into other programs. A module name cannot be the same as a Python keyword. Naming a module as 'for', 'while', etc., will result in an error.

10.3 IMPORTING PYTHON MODULES

The most common way to create a module is to define a separate file containing the code we want to group separately from the rest of the application. *For example*, we might want to create a print routine that an application uses in a number of places. The print routine is not designed to work on its own but is part of the application as a whole. We want to separate it because the application uses it at numerous places and we could potentially use the same code in another application. The ability to reuse code ranks high on the list of reasons to create modules.

Modules also make it easy to reuse the same code in more than one program. If we have written a set of functions that is needed in several different programs, we can place those functions in a module. Then, we can import the module in each program that needs to call one of the functions. There are different ways by which we can import a module. We can use any Python source file as a module by executing an **import** statement.

Standard library of Python is extended as module(s) to a programmer. Definitions from the module can be used within the code of a program. To use these modules in a program, a programmer needs to import the module into other modules or into the main module using any of the following three constructs:

- (i) "import" statement can be used to import a module. It is the simplest and most common way to use modules in our code. It provides access to all attributes (like variables, constants, etc.) and methods or functions present in the module.

```
import <module_name>
```

In case of accessing specific modules, it can be modified as:

```
import module1, module2, ...moduleN
```

To access particular methods from any module, the statement shall be:

```
<module_name>.<function_name>
```

Note: The import statement does not give direct access to the functions. To use/access/invoke a function, we will specify the module name and function name separated by a dot(.). This format is known as **dot notation**.

- (ii) Python's **from** statement lets us import specific attributes or individual objects from a module into the current (calling) module or namespace.

```
from <module_name> import <function_name(s)>
```

Or

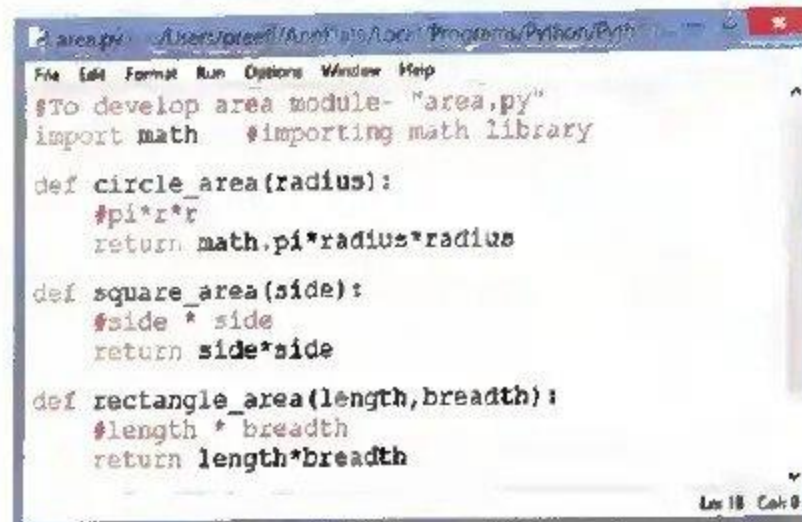
```
from <module_name> import function_name1, ..., function_name2, ...,  
function_nameN
```

- (iii) **import *** statement can be used to import all names from a module into the current calling (namespace) module.

```
from <module_name> import *
```

Let us look at a few simple examples.

Example 1: Suppose we have a Python file named **area.py** which has a few functions in it for calculating the area of a circle, square and rectangle. Use those functions again in some other programs.



```
area.py: C:/Users/oreeti/AppData/Local/Programs/Python/Python36-32/Python36-32/area.py
File Edit Format Run Options Window Help
#To develop area module- "area.py"
import math #importing math library

def circle_area(radius):
    #pi*r*r
    return math.pi*radius*radius

def square_area(side):
    #side * side
    return side*side

def rectangle_area(length,breadth):
    #length * breadth
    return length*breadth
```

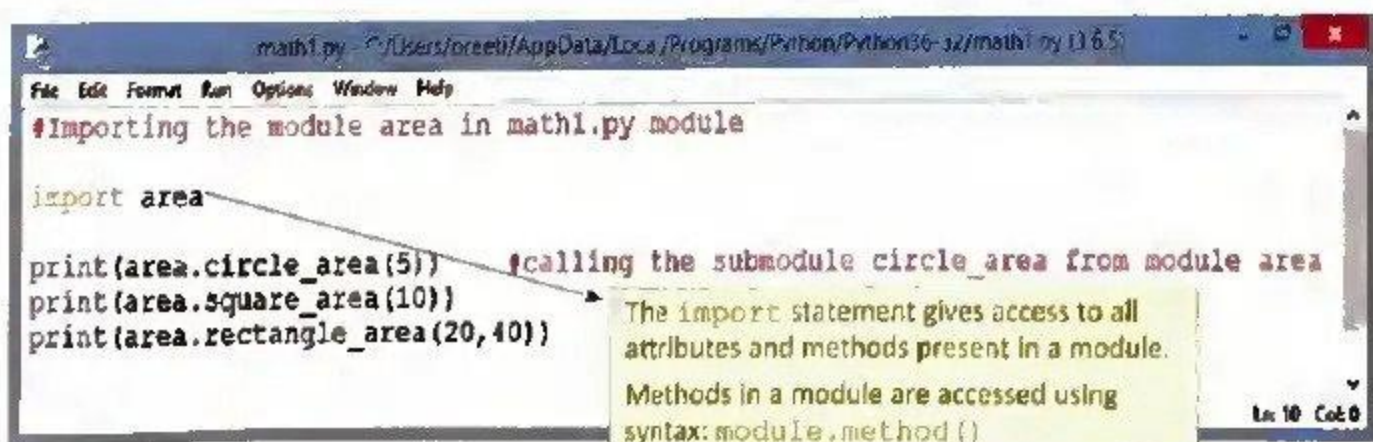
Fig. 10.3(a): area Module

We have created the "area" module (Fig. 10.3(a)) which is a user-created module, not from the Python library.

Note: Always remember that the module area.py should be in the same default root folder where math.py also resides.

However, this program uses the standard math module for using pi() function that belongs to it.

Now, we shall call these modules in another program (Fig. 10.3(b)) named "math1.py".



```
math1.py: C:/Users/oreeti/AppData/Local/Programs/Python/Python36-32/math1.py (3.6.5)
File Edit Format Run Options Window Help
#Importing the module area in math1.py module

import area

print(area.circle_area(5)) #calling the submodule circle_area from module area
print(area.square_area(10))
print(area.rectangle_area(20,40))
```

The import statement gives access to all attributes and methods present in a module. Methods in a module are accessed using syntax: module.method()

Fig. 10.3(b): "math1.py" Module

```

Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:0
7:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for mor
e information.
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/P
ython/Python36-32/math1.py
70.53981633974483
100
800
>>>
Ln 8 Col 4

```

Fig. 10.3(c): Output of Module "math1.py"

When we run this program, the sub-modules, namely `circle_area()`, `square_area()` and `rectangle_area()`, shall be called from module `area` using the statements:

- `area.circle_area()`
- `area.square_area()`
- `area.rectangle_area()`

They shall generate the output as shown in Fig. 10.3(c).

POINT TO REMEMBER

Python is case-sensitive. Therefore, it is advisable that all the module names should be in lower case.

10.3.1 Retrieving Objects from a Module

There is another function `dir()` which, when applied to a module, gives you the names of all that is defined inside the module.

Example 2: To retrieve the listing of all the objects present inside the user-created module "area".

```

Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
re information.
>>> import area
>>> dir(area)
['_builtins_', '_cached_', '_doc_', '_file_', '_loader_', '_name_', '_package_', '_spec_', 'circle_area', 'math', 'rectangle_area', 'square_area']
>>>
Ln 6 Col 4

```

As shown above, in order to obtain information about a module (area in this case), the usage of `dir()` function, with the name of the module to be retrieved as the parameter, shall result in the display of all its respective objects, viz. name of the sub-modules, variables and constants (if any) present in the module.

Example 3: Modification of Example 1—to calculate the area of the circle only. This can be done using the "from" statement along with the name of the sub-module to be called specifically.

```

Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
#math2.py: to calculate area of a circle only
#calling the sub-module from the module "area"

from area import circle_area from allows you to select
specific functions from the
module
print(circle_area(10))

Ln 9 Col 8

```

```

Python 3.6.5 {v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46} [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/math2.py
314.1592653589793
>>>

```

from...import statement is used to import a particular attribute (sub-routine) from a module. In case you do not want the whole of the module to be imported, then you can use from...import statement as we have done in the above example. In order to calculate area of the circle, circle_area() function is being called from module "area".

CTM: For modules having a large number of functions, it is recommended to use **from** instead of **import**.

Now let us see the third format of importing a module using **from...import ***.

The **import *** statement allows a user to import everything from the file or module. This construct will import all Python definitions into the namespace of another module.

However, this construct is not recommended to be used much as it may result in namespace pollution. We may have several objects of the same name and their definitions can be overridden.

Example 4: Modification of Example 3—to import all the sub-modules from module "area".

```

# Use of import * statement

from area import *

print(circle_area(12))
print(square_area(20))
print(rectangle_area(25,15))

```

import * gives access to all functions in area module
You can use all the functions in area module directly

```

RESTART: C:\Users\preeti\AppData\Local\Programs\Python\Python37-32\math3.py
452.3893421169302
400
375
>>>

```

Thus, from the given examples, we have observed that **import** statement in Python can be represented in any of the following for importing a module in some other program:

- (a) **import test:** imports the module 'test' and creates a reference to that module in the current calling program (or namespace). After executing this statement, you can use test.name to refer to the objects defined in module 'test'.
- (b) **import test1, test2, test3:** This statement shall import three modules, namely test1, test2 and test3 in current namespace. As a result, all the definitions of these modules are available and can be used as:

`<module_name>.<name>`

sub-module/any object

- (c) **from test import *:** When you use import * statement, it shall import all the sub-modules or objects defined in the module 'test' into the current namespace. So, after executing this statement, you are required to simply give the name of the object to be referred to, defined in the module test. In other words, <name> would now be used as name only, without specifying module's name (i.e., test.name is not required to be given). If name was already defined, it is replaced by the new version, the one imported from the module.
- (d) **from test import t1,t2,t3:** This statement shall import the module test and create references in the current namespace to the given objects. In other words, you can now use t1, t2, and t3 in your program without specifying their module's name along with them.

Learning Tip: Whenever we are using a module in our program, for that module, its compiled file will be generated and stored in the hard disk permanently.

10.4 MODULE ALIASING

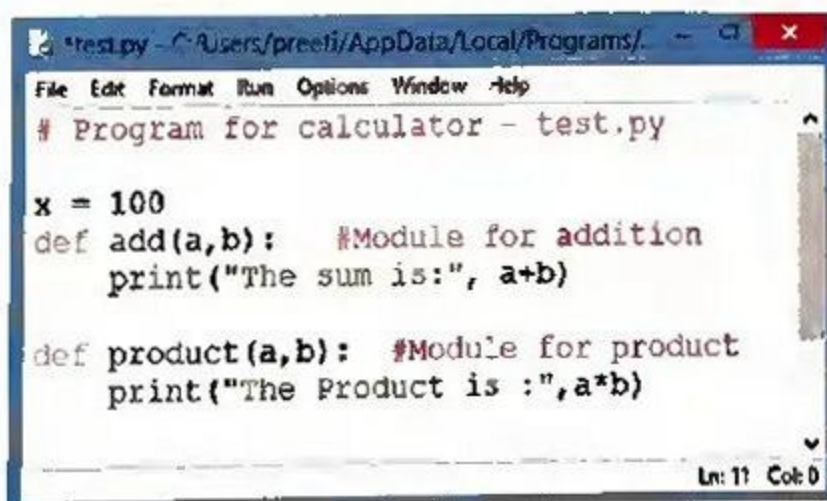
You can create an alias when you import a module by using the **as** keyword. Thus, Python provides the salient feature of renaming a module at the time of import.

Syntax:

```
import <module-name> as <alias_name>
```

Practical Implementation-1

Develop a program calculator for performing addition and multiplication operations using modules and importing these modules in the program.



```
test.py - C:/Users/preeti/AppData/Local/Programs/Python/Python38-64/Python.exe
File Edit Format Run Options Window Help
# Program for calculator - test.py

x = 100
def add(a,b): #Module for addition
    print("The sum is:", a+b)

def product(a,b): #Module for product
    print("The Product is :",a*b)
```

```
prog_test1.py - C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/Python36-32/py
File Edit Format Run Options Window Help
# Program testing
import test
print(test.x) #accessing the value of variable x
test.add(10,20)
test.product(10,20)

>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/Python36-32/py
100
The sum is: 30
The Product is : 200
>>>
```

Ln: 8 Col: 4

Practical Implementation-2

Modify the above program by using alias name for the module "test".

```
prog_test2.py - C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/Python36-32/py
File Edit Format Run Options Window Help
# Program testing using alias module name

import test as m # m is the alias name for module test
print(m.x) #using alias name for accessing the value of variable x
m.add(10,20)
m.product(10,20) #alias name for calling the functions of the module

>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/Python36-32/py
100
The sum is: 30
The Product is : 200
>>>
```

Ln: 11 Col: 0

Ln: 12 Col: 4

As is evident from the same output obtained from the above two programs, we can say that alias name works well with Python modules.

Here, **test** is the original module name and **m** the alias name. We can easily access members of the main module by using the alias name **m**.

10.5 MEMBER ALIASING

Like module aliasing, member aliasing is also supported and implemented in Python modules.

Practical Implementation-3

Modify the previous program for implementing aliasing with the members of the module.

```
prog_test3.py - C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/Python36-32/py
File Edit Format Run Options Window Help
# Member Aliasing

from test import x as p, add as sum
print(p)
sum(100,200)

>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/Python36-32/py
100
The sum is: 300
>>>
```

Ln: 17 Col: 4

Once we have defined an alias name, we should use alias name only and use of original module name should be avoided as it may create ambiguity and result in an unexpected output or sometimes even an error, as shown below:



```
File Edit Shell Debug Options Window Help
RESTART: C:/Users/preeti/AppData/Local
/Programs/Python/Python36-32/prog_test3
.py
100
The sum is: 300
>>> from test import x as p
>>> print(x)
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <modul
e>
    print(x)
NameError: name 'x' is not defined
>>>
```

Practical Implementation-4

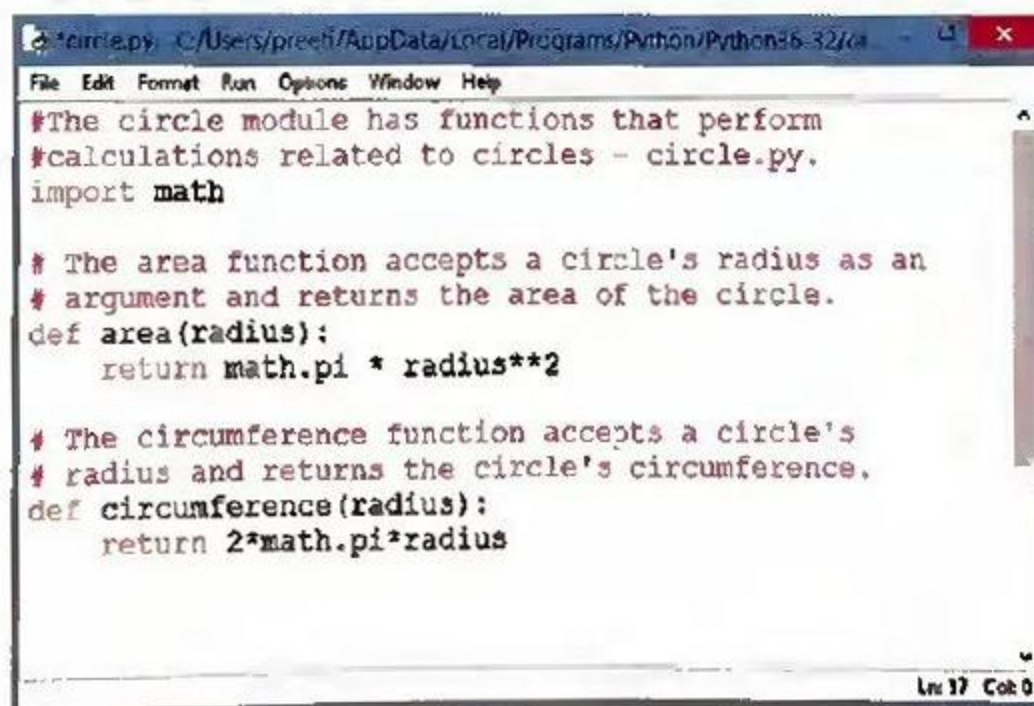
Write a program in Python that calculates the following:

- Area of a circle
- Circumference of a circle
- Area of a rectangle
- Perimeter of a rectangle

Create respective modules for each of the operations and call them separately using a menu-driven program.

There are obviously two categories of calculations required in this program: those related to circles, and those related to rectangles. You could write all circle-related functions in one module and all rectangle-related functions in another module.

circle Module



```
*circle.py C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/
File Edit Format Run Options Window Help
#The circle module has functions that perform
#calculations related to circles - circle.py.
import math

# The area function accepts a circle's radius as an
# argument and returns the area of the circle.
def area(radius):
    return math.pi * radius**2

# The circumference function accepts a circle's
# radius and returns the circle's circumference.
def circumference(radius):
    return 2*math.pi*radius

Ln: 17 Col: 0
```

rectangle Module

```
File Edit Format Run Options Window Help
#the Rectangle module -The rectangle module has functions that perform
# calculations related to rectangles.
# The area function accepts a rectangle's width and
# length as arguments and returns the rectangle's area.

def area(width, length):
    return width * length
# The perimeter function accepts a rectangle's width
# and length as arguments and returns the rectangle's
# perimeter.

def perimeter(width, length):
    return 2 * (width + length)
```

Ln: 16 Col: 9

```
File Edit Format Run Options Window Help
#Main Program importing modules
#This program allows the user to choose various
# geometrical calculations from a menu. This program
# imports the circle and rectangle modules.

import circle
import rectangle
choice = 0
ch = "y"
# The display-menu function displays a menu.
#def displaymenu():
while (ch == "y"):
    print("MENU")
    print("1. Area of a circle")
    print("2. Circumference of a circle")
    print("3. Area of a rectangle")
    print("4. Perimeter of a rectangle")
    print("5. Quit")
    choice = int(input("Enter your choice :"))
    if (choice == 1):
        radius = int(input("Enter the circle's radius: "))
        print("The area is", circle.area(radius))
    elif (choice == 2):
        radius = int(input("Enter the circle's radius: "))
        print("The circumference is", circle.circumference(radius))
    elif (choice == 3):
        width = int(input("Enter the rectangle's width: "))
        length = int(input("Enter the rectangle's length: "))
        print('The area is', rectangle.area(width, length))
    elif (choice == 4):
        width = int(input("Enter the rectangle's width: "))
        length = int(input("Enter the rectangle's length: "))
        print('The perimeter is', rectangle.perimeter(width, length))
    elif (choice == 5):
        print('Exiting the program ...')
    else:
        print('Error: invalid selection.')
```

Ln: 40 Col: 4

```
File Edit Shell Debug Options Windows Help
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python37-32/prog mod alt1.py
MENU
1. Area of a circle
2. Circumference of a circle
3. Area of a rectangle
4. Perimeter of a rectangle
5. Quit
Enter your choice :2
Enter the circle's radius: 4
The circumference is 25.132741228718345
MENU
1. Area of a circle
2. Circumference of a circle
3. Area of a rectangle
4. Perimeter of a rectangle
5. Quit
Enter your choice :3
Enter the rectangle's width: 55
Enter the rectangle's length: 66
The area is 3630
Ln 46 Col 22
```

```
MENU
1. Area of a circle
2. Circumference of a circle
3. Area of a rectangle
4. Perimeter of a rectangle
5. Quit
Enter your choice :4
Enter the rectangle's width: 20
Enter the rectangle's length: 42
The perimeter is 124
MENU
1. Area of a circle
2. Circumference of a circle
3. Area of a rectangle
4. Perimeter of a rectangle
5. Quit
Enter your choice :1
Enter the circle's radius: 10
The area is 314.1592653589793
MENU
1. Area of a circle
2. Circumference of a circle
3. Area of a rectangle
4. Perimeter of a rectangle
5. Quit
Enter your choice :5
```

10.6 LOCATING MODULES

When you import a module, the Python interpreter searches for the module in the following sequence:

- The current directory.
- If the module isn't found, Python then searches each directory in the shell variable PYTHONPATH.
- If all else fails, Python checks the default path, which is the location where Python has been installed on your computer system.

Thus, PYTHONPATH is the **variable** that tells the interpreter where to locate the module files imported into a program. Hence, It must include the Python source library directory and the directories containing Python source code. You can manually set PYTHONPATH but usually the Python installer will preset it.

10.7 STANDARD BUILT-IN PYTHON MODULES

In Python, we have worked with script mode so far, which provides the capability to retain our work for future usage. For working in script mode, we need to write a module/function in Python and save it in the file having .py extension.

A Python module is written once and used/called as many times as required.

CTM: Modules are the most important building blocks for any application in Python and work on divide and conquer approach.

Modules can be categorized into the following two types:

- (i) Built-in Modules
- (ii) User-defined Modules

Let us discuss built-in modules in detail.

10.7.1 Built-in Functions

Built-in functions are predefined functions that are already available in Python. Functions provide efficiency and structure to a programming language. Python has many useful built-in functions to make programming easier, faster, and more powerful. These are always available in the standard library and for using them, we don't have to import any module (file).

Let us discuss math and datetime modules which are one of the most commonly used built-in modules in Python.

To access/use any of the functions present in the imported module, you have to specify the name of the module followed by the name of the function, separated by a dot (also known as a period). This format is called **dot notation**.

For example,

```
import math  
result = math.sqrt(64)    #Gives output as 8 which gets  
                           stored in variable 'result'
```

sqrt() is one of the functions that belongs to math module. This function accepts positive integer as an argument and returns the square root of the argument. Thus, the above statement calls the sqrt() passing 64 as an argument and returns the square root of the number passed as the argument, i.e., 8, which is then assigned to the variable 'result'.

math Module

Let us understand a few more functions from the math module.

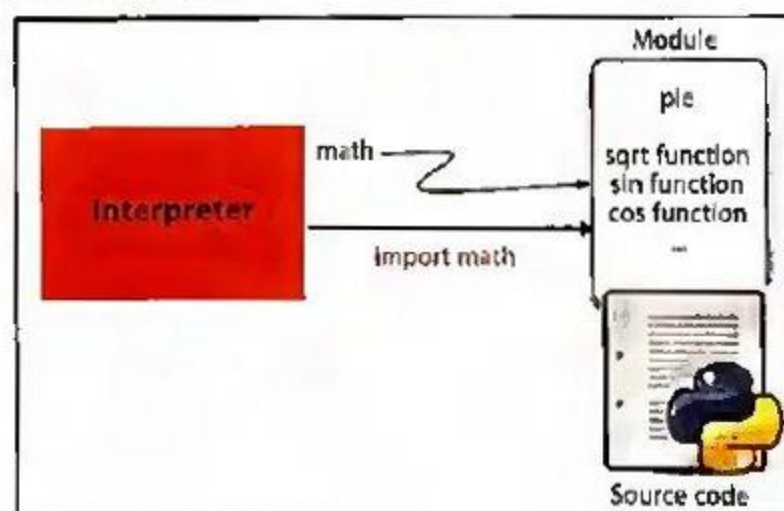


Fig. 10.4: math Module In Python

It contains different types of mathematical functions. Most of the functions in this module return a float value. In order to use the math module, we need to import it using the following statement:

```
import math #the very first statement to be given
```

Let us discuss commonly-used functions in math module.

- **pi:** It is a mathematical constant, the ratio of the circumference of a circle to its diameter (3.14159...)

For example,

```
>>> print("The value of pi is :", math.pi)
The value of pi is : 3.141592653589793
```

- **e:** It is a mathematical constant that returns e raised to the power x, where $e = 2.718281$. It is the base of natural logarithms. It is also called Euler's number.

For example,

```
>>> print("The value of e is :", math.e)
The value of e is : 2.718281828459045
```

- **ceil(x):** Returns the smallest integer that is greater than or equal to x.

For example,

```
>>> print("math.ceil(3.4):", math.ceil(3.4))
Output:      math.ceil(3.4): 4
```

- **floor(x):** Returns the largest integer that is less than or equal to x.

```
>>> math.floor(-45.17)
-46
>>> math.floor(100.12)
100
>>> math.floor(100.72)
100
```

- **pow(x,y):** It returns the value of x^y , where x and y are numeric expressions.

```
>>> print("math.pow(3, 3):", math.pow(3, 3))
math.pow(3, 3): 27.0
>>> math.pow(100, -2)
0.0001
>>> math.pow(2, 4)
16.0
>>> math.pow(3, 0)
1.0
```

- **sqrt(x):** Returns the square root of x.

```
>>> print("math.sqrt(65):", math.sqrt(65))
math.sqrt(65): 8.06225774829855
>>> math.sqrt(100)
10.0
>>> math.sqrt(7)
2.6457513110645907
```

Alternatively,

```
>>> print("math.sqrt(65):", round(math.sqrt(65), 2))
math.sqrt(65): 8.06
```

☛ **fabs(x):** Returns the absolute value of x, represented as-
`math.fabs(x)`

where, x can be any numeric value.

For example,

```
>>> print(math.fabs(500.23))
500.23
>>> print(math.fabs(-200))
200
>>> print(math.fabs(-15.6343))
15.6343
```

☛ **log10(x):** Returns the base-10 logarithm of x.

```
>>> print("math.log10(100):", math.log10(100))
math.log10(100): 2.0
>>> math.log10(100.12)
2.0005208409361854
>>> math.log10(100.72)
2.003115717099806
```

☛ **cos(x):** Returns the cosine of x in radians.

```
>>> math.cos(3)
-0.9899924966004454
>>> math.cos(-3)
-0.9899924966004454
>>> math.cos(0)
1.0
>>> math.cos(math.pi)
-1.0
```

☛ **sin(x):** Returns the sine of x in radians.

```
>>> math.sin(3)
0.14112000806
>>> math.sin(-3)
-0.14112000806
>>> math.sin(0)
0.0
```

☛ **tan(x):** Returns the tangent of x in radians.

```
>>> math.tan(3)
-0.1425465430742778
>>> math.tan(-3)
0.1425465430742778
>>> math.tan(0)
0.0
```

☛ help function

If you forget how to use a standard function, Python's library utilities can help. In this case, Python's `help()` library function is extremely relevant. It is used to know the purpose of a function and how it is used.

For example,

```
>>> import math
>>> print(help(math.cos))
Help on built-in function cos in module math:
cos(x, /)
Return the cosine of x (measured in radians)
None
```

random Module (Generating Random Numbers)

The programming concepts we have learnt so far involved situations where we were aware of the definite output to be obtained when a particular task was to be executed, which is described as a deterministic approach. But there are certain situations/applications that involve games or simulations which work on a non-deterministic approach. In these types of situations, random numbers are extensively used such as:

1. Pseudorandom numbers on lottery scratch cards.
2. Recaptcha (like in login forms) uses a random number generator to define which image is to be shown to the user.
3. Computer games involving throwing of a dice, picking a number, or flipping a coin.
4. Shuffling deck of playing cards, etc.

In Python, random numbers are not enabled implicitly; therefore, we need to invoke random module explicitly or to invoke random code library in order to generate random numbers. This module contains functions that are used for generating random numbers.

import statement is the first statement to be given in a program for generating random numbers:

```
import random
```

The various functions associated with this module are as follows:

- ☛ **randrange():** This method generates an integer between its lower and upper argument. By default, the lower argument is 0.

For example,

```
ran_number = random.randrange(30)
```

Alternatively,

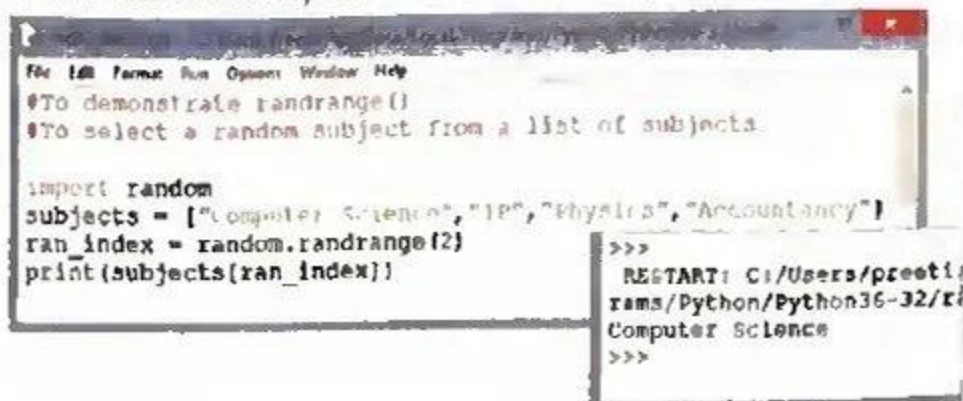
```
>>> ran_number = random.randrange(30)
>>> print(ran_number)
1
>>>
```

```
>>> import random
>>> random.randrange(30)
13
>>>
```

This line of code shall generate random numbers from 0 to 29.

Let us take another example:

Example 5: To select a random subject from a list of subjects.



```
File Edit Format Run Options Window Help
#To demonstrate randrange()
#To select a random subject from a list of subjects

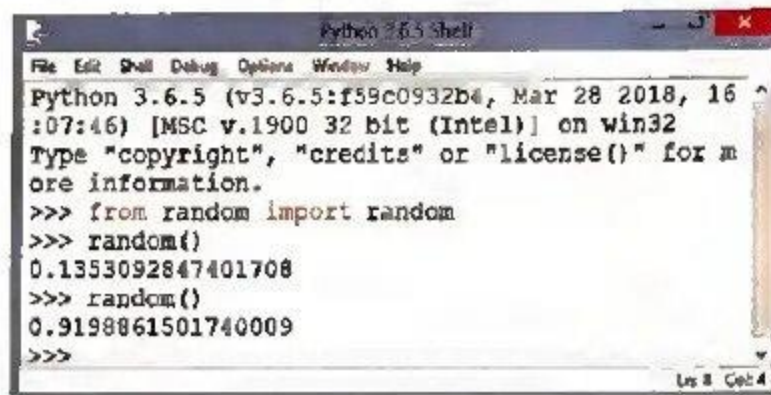
import random
subjects = ["Computer Science", "IP", "Physics", "Accountancy"]
ran_index = random.randrange(2)
print(subjects[ran_index])

>>>
RESTART: C:/Users/preeti/
ams/Python/Python36-32/xl
Computer Science
>>>
```

random()

This function generates a random number from 0 to 1 such as 0.5643888123456754. This function can be used to generate pseudorandom floating point values. It takes no parameters and returns values uniformly distributed between 0 and 1 (including 0, but excluding 1). Let us take some examples to gain better clarity about this function:

Example 6:



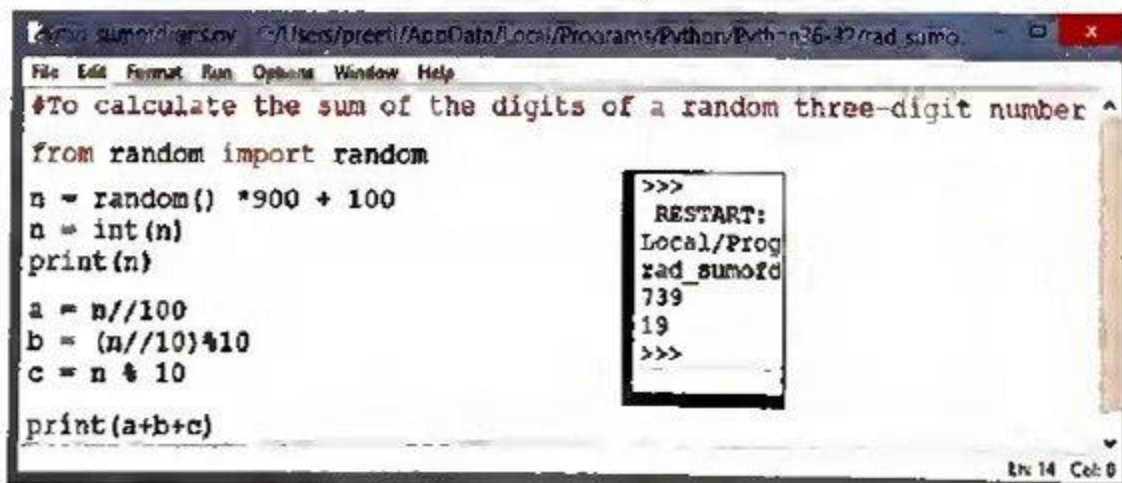
```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16
:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for m
ore information.
>>> from random import random
>>> random()
0.1353092847401708
>>> random()
0.9198861501740009
>>>
```

POINT TO REMEMBER

This is to be kept in mind that every time we are executing this program, it shall display a different value as random() method always picks up any random value from the given range.

As is evident from the above statements, each time random() generates a different number, the random() function can also be written with the following alternative approach:

Example 7: Program to calculate the sum of the digits of a random three-digit number.



```
File Edit Format Run Options Window Help
#To calculate the sum of the digits of a random three-digit number
from random import random
n = random() * 900 + 100
n = int(n)
print(n)
a = n//100
b = (n//10)%10
c = n % 10
print(a+b+c)

>>>
RESTART:
Local/Prog
rad_sumofd
739
19
>>>
```

Explanation:

The random() function generates a random fractional number between 0 and 1. When a random number generated using random() gets multiplied by 900, a random number is obtained between 0 and 899. When you add 100 to it, you get a number between 100 and 999.

Then, the fractional part gets discarded using `int()` and gets printed on the shell window. In the next statement, the first digit (the most significant) of the number is extracted by dividing it with 100 ($a = n//100$).

The last digit of the number is extracted by dividing it by 10. The remaining number then gets divided by 10 and the quotient is again mod with 10 to get ten's digit and the remainder is again mod with 10 and this process gets repeated till all the digits are extracted and finally added together. In the last statement, the sum of digits is calculated and displayed on the screen.

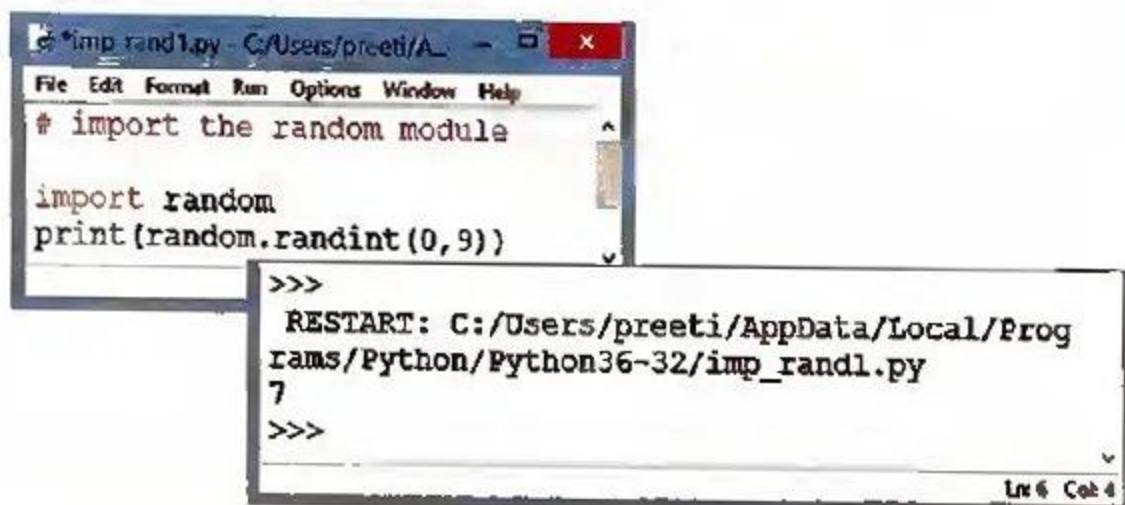
• `randint()`:

This function accepts two parameters, `a` and `b`, as the lowest and highest number; returns a number 'N' in the inclusive range `[a,b]`, which signifies $a \leq N \leq b$, where the endpoints are included in the range. This function generates a random integer number between two given numbers. This function is best suited for guessing number applications. The syntax is:

```
random.randint(a,b)
```

Here, `a` is the lower bound and `b` is the upper bound. In case you input a number `N`, then the result generated will be $a \leq N \leq b$. Please make sure that the upper limit is included in `randint()` function.

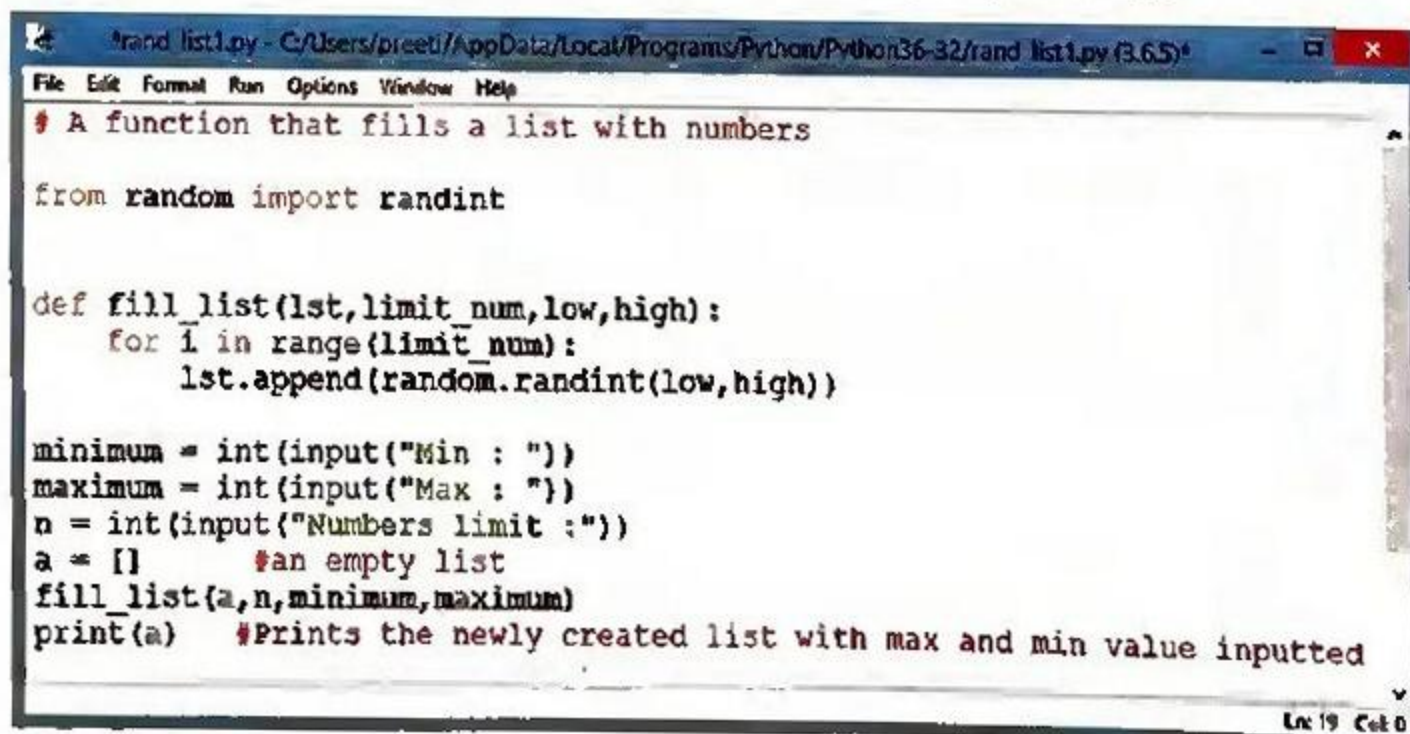
Example 8: To generate a number between 0 and 9.



```
*imp_rand1.py - C:/Users/preeti/A...
File Edit Format Run Options Window Help
# import the random module
import random
print(random.randint(0,9))

>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/imp_rand1.py
7
>>>
```

Example 9: Write a function that fills a list with numbers. (Using `randint()`)



```
*rand_list1.py - C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/rand_list1.py (3.6.5)
File Edit Format Run Options Window Help
# A function that fills a list with numbers
from random import randint

def fill_list(lst, limit_num, low, high):
    for i in range(limit_num):
        lst.append(random.randint(low, high))

minimum = int(input("Min : "))
maximum = int(input("Max : "))
n = int(input("Numbers limit : "))
a = [] #an empty list
fill_list(a,n,minimum,maximum)
print(a) #Prints the newly created list with max and min value inputted
```



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018,
16:07:46) [MSC v.1900 32 bit (Intel)] on win3
2
Type "copyright", "credits" or "license()" fo
r more information.
>>>
RESTART: C:/Users/preeti/AppData/Local/Progr
ams/Python/Python36-32/rand_list1.py
Min : 10
Max : 50
Numbers limit :5
[33, 25, 10, 13, 20]
>>>
```

statistics Module

The statistics module implements many common statistical formulas for efficient calculations using Python's various numerical types (int, float, Decimal, and Fraction).

It provides functions for calculating mathematical statistics of numeric (Real-valued) data. Before using the functions of this module, it is required to be imported first using the statement—`import statistics`.

The following popular statistical functions are defined in this module:

- **mean():** The `mean()` function calculates the arithmetic mean of the numbers in a list.

For example,

```
>>> import statistics
>>> statistics.mean([4, 6, 7, 8, 10, 45])
13.333333333333334
```

- **median():** The `median()` function returns the middle value of numeric data in a list. This function finds the centre value, and if the data set has an even number of values, it averages the two middle items.

For example,

```
>>> import statistics
>>> statistics.median([1, 2, 3, 8, 9])
3
>>> statistics.median([1, 2, 3, 7, 8, 9])
5.0
```

- **mode():** This function returns the most repeated value of the list/sequence passed. Its syntax is:

```
statistics.mode(x)
```

For example,

```
>>> import statistics
>>> statistics.mode([21, 24, 21, 45, 56, 67, 21])
21
>>> statistics.mode(("red", "blue", "red"))
'red'
```

Example 10: Write a program using a user-defined function `cal_Mean()` to calculate the mean of floating values stored in a list.

Function to calculate mean

The requirements are listed below:

1. The function should have 1 parameter (list containing floating point values).
2. To calculate mean by adding all the numbers and dividing by total number of elements.

```
prog_mean_cal.py - C:/Users/preeti/AppData/Local/Programs/Python/Python31-32/prog_mean_cal.py (3...
File Edit Format Run Options Windows Help
#Program using a user defined function cal_Mean() to calculate the mean
#of floating values stored in a list

def cal_Mean(list1): #function to compute means of values in list
    total = 0
    count = 0
    for i in list1:
        total = total + i #Adds each element i to total
        count = count + 1 #Counts the number of elements
    mean = total/count #mean is calculated
    print("The calculated mean is:",mean)

list1 = [2.6,3.4,8.5,7.9]
#Function call with list "list1" as an argument
cal_Mean(list1)
```

```
>>>
RESTART: C:/Users/preeti/AppData
1.py
The calculated mean is: 5.6
```

POINTS TO REMEMBER

- import statement can be written anywhere in the program.
- Module must be imported only once.
- In order to get a list of modules available in Python, we can use the following statement:
`>>> help("module")`
- To view the content of a module say `math`, type the following:
`>>> help("math")`

The modules in the standard library can be found in the Lib folder of Python.



MEMORY BYTES

- A module is a file containing Python code. A package, however, is like a directory that holds sub-packages and modules.
- A package must hold the file `__init__.py`. This does not apply to modules.
- The Python standard library is an extensive collection of functions and modules that help the programmer in the faster development of programs.
- A module is a Python file that contains definitions of multiple functions.
- To import everything from a module, we use the wild card `*`. But this does not work with packages.
- A module is a separately saved unit whose functionality can be reused at will.
- A Python module can contain objects like docstrings, variables, constants, classes, objects, statements, functions and has the `.py` extension.



- A Python module can be imported in a program using import statement.
- Irrespective of the number of times a module is imported, it is loaded only once.
- To import specific functions in a program from a module, from statement can be used.
- There are two forms of import statements
 - import <modulename>
 - from <module> import <object>
- The random module of Python provides random number generation functionality.
- A math module of Python provides math functionality.
- By default, Python names the segment with top-level statements (main program) as `_main_`.
- The `mean()` method calculates the arithmetic mean of the numbers in a list.
- The `mode()` function returns the most repeated value of the list/sequence passed.
- The statistics module implements many common statistical formulas for efficient calculations.

OBJECTIVE TYPE QUESTIONS

1. Fill in the blanks.

- (a) A Python is a collection of modules and packages.
- (b) A is a Python file that constitutes only definitions of variables, functions and classes.
- (c) The statement reads a module file and makes its contents available for use.
- (d) module in Python provides mathematical functions.
- (e) function returns the smallest integer value greater than or equal to a floating point number.
- (f) function returns a set of data values that appears most often in a data set.
- (g) The output of the following statement: `X = statistics.mean([2,15,16,19])` is
- (h) To access a method/function from a module, we have to specify the name of the function separated by a operator.
- (i) A module can be classified as either or
- (j) A module constitutes of 2 sub-components—executable as well as
- (k) function is used to get all information about module, i.e., name of all functions, variables, etc., available in that module.

2. State whether the following statements are True or False.

- (a) A module line (header) is written only once at the top of the program.
- (b) A function call can be made several times in a Python module.
- (c) A module is loaded only once regardless of the number of times it is used.
- (d) The statement to import math module is—call math.
- (e) A module is not necessarily to be imported before being used.
- (f) `floor()` returns the largest integer value less than or equal to a floating point number.
- (g) `fabs()` returns the same number if it is a negative value.
- (h) `math.sqrt(x)` returns a `ValueError` if `x` is a negative number.
- (i) The `mean()` method calculates the arithmetic mean of the numbers in a list.
- (j) Each time we use the `randint()` function, it produces different results.

3. Multiple Choice Questions (MCQs):

- (a) What is the output of `math.ceil(4.4)`?

(i) 5	(ii) 4	(iii) 4.0	(iv) 5.0
-------	--------	-----------	----------
- (b) What will be the output on screen after executing: `print(math.fabs(-6.4))`?

(i) -6.4	(ii) 6.4	(iii) 4	(iv) -6
----------	----------	---------	---------

- (c) If $a, b, c = 3, 4, 1$ then what will be the value of `math.sqrt(b)*a - c`?
- (i) 5.0 (ii) 5 (iii) 2 (iv) 4.0
- (d) What is the value of x , if `x = math.factorial(0)`?
- (i) 1 (ii) 0 (iii) error (iv) None of these
- (e) What is the value of x , if `x = math.sqrt(25.0)`?
- (i) (5,-5) (ii) 5 (iii) 5.0 (iv) (5.0, -5.0)
- (f) What value will be returned, if `x = math.floor(-24.6)`?
- (i) 24.6 (ii) -24 (iii) -25 (iv) -24.6
- (g) What value will be displayed on executing the following arithmetic expression
`x = math.pow(0, -4)`?
- (i) 1 (ii) 16 (iii) DomainError (iv) None of these
- (h) What output will be displayed after executing the following code?
`statistics.median([11,22,33,44,55,66])`
- (i) 28.5 (ii) 38.5 (iii) 5.5 (iv) 6.5
- (i) On executing the following code, what output will be displayed?
`statistics.mode([2.5,3.2,3.3,2.5,8.1,9.9,2.5,5.1])`
- (i) 2.5 (ii) 3.2 (iii) 5.1 (iv) 8.1
- (j) What will the following code display on execution?
`X = statistics.mean([2,15,6,19])`
- (i) 10.5 (ii) 10.6 (iii) Error (iv) None of these

SOLVED QUESTIONS

- Python has certain functions that you can readily use without having to write any special code. What types of functions are these?
Ans. The predefined functions that are always available for use are known as Python's built-in functions. Examples of some built-in functions are: `len()`, `type()`, `int()`, `input()`, etc.
- What is a Python module? What is its significance?
Ans. A "module" is a chunk of Python code that exists in its own (.py) file and is intended to be used by Python code outside itself. Modules allow one to bundle together code in a form in which it can be easily used later. Modules can be "imported" in other programs so that the functions and other definitions in imported modules become available to code that imports them.
- What are docstrings? How are they useful?
Ans. A docstring is just a regular Python triple-quoted string that is the first thing in a function body/a module/a class. When executing a function body (or a module/class), the docstring doesn't do anything like comment, but Python stores it as part of the function documentation. This documentation can later be displayed using the `help()` function.
So, even though docstrings appear like comments (no execution), they are different from comments.
- What happens when Python encounters an import statement in a program? What would happen if there is one more import statement for the same module, already imported in the same program?
Ans. When Python encounters an import statement, it does the following:
 - Code of the imported module is interpreted and executed.
 - Defined functions and variables created in the module are now available to the program that imported the module.
 - For imported module, a new namespace is set up with the same name as that of the module.
 Any duplicate import statement for the same module in the same program is ignored by Python.
- What is an argument? Give an example.
Ans. An argument is data passed to a function through function call statement. For example, in the statement `print(math.sqrt(25))`, the integer 25 is an argument.



6. What is the utility of built-in function `help()`?

Ans. Python's built-in function `help()` is very useful. When it is provided with a program name or a module name or a function name as an argument, it displays the documentation of the argument as help. It also displays the docstring within its passed-argument's definition.

For example,

```
>>> help(math)
```

will display the documentation related to module `math`.

It can even take function name as argument.

For example,

```
>>> help(math.sqrt())
```

The above code will list the documentation of `math.sqrt()` function only.

7. Write a function `compute_volume()` that:

- (i) Asks the user to input a diameter of a sphere (centimetres, inches, etc.)
- (ii) Sets a variable called `radius` to one-half of that number.
- (iii) Calculates the volume of a sphere using this radius and the formula:
 $\text{Volume} = \frac{4}{3} \pi r^3$
- (iv) Prints a statement estimating that this is the volume of the sphere; include the appropriate unit information in litres or quarts. Note that there are 1000 cubic cm in a litre and 57.75 cubic inches in a quart.
- (v) Returns this same amount as the output of the function.

Ans. `def compute_volume():`

```
import math
diameter = float(input("Please enter the diameter in inches:"))
radius = diameter/2
Volume_in_cubic_inches = ((4/3) * math.pi) * (radius ** 3)
Volume_in_quarts = Volume_in_cubic_inches/57.75
print("The sphere contains", Volume_in_quarts, 'quarts')
return(Volume_in_quarts)
```

8. Rohit is a Python programmer and has imported the module `math`. From this `math` module he wants to use the function `sqrt()` to calculate square root of `n`. But he has forgotten how to use function from imported module. Help Rohit use the function `sqrt()`.

```
import math
n = int(input("Enter number "))
num = ..... # statement to call sqrt() function for value n
print(num)
```

Ans. `num = math.sqrt(n)`

9. Name any two built-in modules and two functions belonging to each category.

Ans. Built-in modules:

- (i) `math: sqrt(), pow()`
- (ii) `statistics: mean(), median()`

10. What will be the output of the following code?

```
def interest(prnc, time=2, rate = 0.10):
    return (prnc * time * rate)
print(interest(6100, 1))
print(interest(5000, rate = 0.05))
print(interest(5000, 3, 0.12))
print(interest(time = 4, prnc = 5000))
```

Ans. 610.0
500.0
1800.0
2000.0

11. (a) What are the possible numbers generated from the Python code below:

```
import random as r
num = r.randrange(1, 5, 2)
print(num)
```

Ans. Either 1 or 3

(ii) What will be the output of the following code?

```
import sys
print(sys.argv)
```

- (i) ['Python', 'code', 'run', 'on', 'page']
- (ii) ['Python', 'code', 'run', 'on', 'page', '1']
- (iii) ['Python', 'code', 'run', 'on', 'page', '1', '2']
- (iv) ['Python', 'code', 'run', 'on', 'page', '1', '2', '3']
- (v) ['Python', 'code', '1', '2']
- (vi) ['Python', 'code', '1', '2', '3', '4', '5', '6', '7', '8', '9', '10']

Ans. (i) ☐ (ii) ☐
 (iii) ☐ (iv) ☐
 (v) ☐ (vi) ☐

12. What is the utility of Python standard library's math module and random module?

Ans. (i) **Math module:** The math module is used for math-related functions that work with all number types except for complex numbers.
 (ii) **Random module:** The random module is used for different random number generator functions.

13. What is a module? Give at least two reasons why you need modules.

Ans. A module is a file containing Python definitions and statements. The code modules consist of the following salient features:

1. A module allows code reuse.
2. It makes the "main" program shorter and more readable.
3. It enables clearer code organization.

14. Go through the Python code shown below and find out the possible output or outputs from the suggested output options (i) to (iv). Also, write the minimum and maximum values, which can be assigned to the variable MyNum

```
import random
MyNum=1
Max=5
MyNum=random.randint(0,Max)
for i in range(MyNum,20):
    print(i, " ", end=" ")
```

Ans. (i) 16 517 518 519
 Minimum value: 1, Maximum value: 20

15. "Python has certain functions that you can readily use without having to write any special code." What type of functions are these? [HOTS]

Ans. The pre-defined functions that are always available for use are known as Python's built-in functions. For example:
 len(), type(), int(), input(), etc.

16. What is the utility of built-in function help()?

Ans. Python's built-in function help() is very useful. When it is provided with a program name or a module name or a function name as an argument, it displays the documentation of the argument as help. It also displays the details within its passed argument's definition. For example:
 help(math) will display the documentation related to module math.

17. What is the utility of Python standard library's math module and random module?

Ans. Math module is used for math-related functions that work with all number types except for complex numbers, random module is used for different random number generator functions.

18. Observe the following code and answer the question based on it.

```
# the math operation module
def add(a,b):
    return a + b
def subtract(a,b):
    return a - b
```

Fill in the blanks for the following code:

1. (_____) math_operation
get the name of the module.
2. print (_____)
output: math_operation
Add 1 and 2
3. print (_____ (1, 2))
output 3

Ans. 1. import
2. math_operation.__name__
3. math_operation.add

19. Consider the code given in Q.18 and on the basis of it, complete the code given below:

```
# import the subtract function
# from the math_operation module
1. _____
# subtract 1 from 2
2. print( _____ (2, 1) )
# output : 1
# Import everything from math_operation
3. _____
print(subtract (2, 1))
# output: 1
print(add (1, 1))
# output : 2
```

Ans. 1. from math_operation import subtract
2. subtract
3. from math_operation import *

20. Why is the use of import all statements not recommended?

Ans. While using import all statements, we can overwrite functions as there may have several objects of the same name, especially if we don't maintain that module. It may also result in namespace pollution.

21. What is a Python module?

Ans. A module is a script in Python that defines import statements, functions, classes, and variables. It also holds executable Python code. ZIP files and DLL files can also be modules. The module holds its name as a string that is in a global variable.

22. Define 'module' and 'package'.

[HOTS]

Ans. Each Python program file is a 'module' which imports other modules like 'objects' and 'attributes'. A Python program folder is a 'package' of 'modules'. A package can have 'modules' or 'sub-folders'.

23. Why is a banner saying "RESTART" always displayed in Python module/program execution?

Ans. When you execute a program from the IDLE Editor, the interpreter gives a banner saying "RESTART", meaning that all the things you defined in any shell session so far are wiped clean and the program you are running starts afresh.

24. What is the output of the following piece of code?

```
#mod1
def change(a):
    b=[x*2 for x in a]
    print(b)
#mod2
```

```
def change(a):
    b=[x*x for x in a]
    print(b)
from mod1 import change
from mod2 import change
#main
s=[1,2,3]
change(s)
```

Note: Both the modules mod1 and mod2 are placed in the same program.

- (a) [2,4,6]
- (b) [1,4,9]
- (c) [2,4,6], [1,4,9]
- (d) There is a name clash

Ans. (d)

Explanation: A name clash is when two different entities with the same identifier become part of the same scope. Since both the modules have the same function name, there is a name clash.

25. What is the output of the following piece of code?

```
from math import factorial
print(math.factorial(5))
```

- (a) 120
- (b) Nothing is printed
- (c) Error, method factorial doesn't exist in math module
- (d) Error, the statement should be: print(factorial(5))

Ans. (d)

Explanation: In the "from-import" form of import, the imported identifiers (in this case factorial()) aren't specified along with the module name.

26. Which of the following is not an advantage of using a module?

- (a) It provides a means of reusing program code.
- (b) It provides a means of dividing up tasks.
- (c) It provides a means of reducing the size of the program.
- (d) It provides a means of testing individual parts of the program.

Ans. (c)

Explanation: The total size of the program remains the same regardless of whether modules are used or not. Modules simply divide the program.

27. Consider the set of height measurements (in feet) of students in a class:

Student	Height
Rishabh	5.9
Anant	5.5
Rinku	6.1
Tapish	6.0
Ajay	7.2

Write a Python program to calculate the average height of the students in the class and median value using Statistics module.

Ans. # To calculate the average height of the students in a class

```
import statistics
heights = [5.9, 5.5, 6.1, 6.0, 7.2] # the height data
avg = statistics.mean(heights)
print("Average height in the class:", avg)
median_value = statistics.median(heights)
print("Median value for heights:", median_value)
```

Output:

Average height in the class: 6.14

Median value for heights: 6.0



Ans. import random

Explanation: In the above code, the user is asked to guess a number, which is then fetched and gets stored inside the variable "guess_num". If the user's guess is wrong, then the prompt appears again and the user continues to input another number repetitively until the guess is correct. On successful guess, the user will get a "Well guessed!" message, and the program will exit.

- What is a Python library? Explain with example.
- Write a module to input total number of days and find the total number of months and remaining days and display it in another program.
- What is a module? What is the file extension of a Python module?
- How do you reuse the functions defined in a module in your program?
- In how many ways can you import objects from a module?
- How are the following two statements different from each other?
(i) `import math` (ii) `from math import *`
- What is the utility of math module?
- How does Python resolve the scope of a name or identifier?
- A program having multiple functions is considered better designed than a program without any functions. Why?
- Write a module called `calculate_area()` that takes base and height as an input argument and returns an area of a triangle as output. The formula used is
 $\text{Triangle Area} = \frac{1}{2} * \text{base} * \text{height}$
- Rewrite the following Python code after removing all syntax error(s). Underline the corrections done.

12. How is `math.ceil (89.7)` different from `math.floor (89.7)`?
13. Out of `random()` and `randint()`, which function should we use to generate random numbers between 1 and 5. Justify.
14. What is the difference between `import` statement and `from import` statement?
15. Why is `from import*` statement not recommended for importing Python modules in an external program?
16. Write a function to calculate volume of a box with appropriate default values for its parameters. Your function should have the following input parameters:
 - (a) Length of box
 - (b) Width of box
 - (c) Height of boxTest it by writing the complete program to Invoke it.
17. Consider the amount of donations received by a charitable organization given as under:
donations = [100, 60, 70, 900, 100, 200, 500, 500, 503, 600, 1000, 1200]
Now write a Python program to calculate the average amount obtained and median of the above data.
18. Write a Python program to generate a random number between 0 and 9.
19. Define a function which will print Fibonacci series till n passed as argument to the function.

20. Consider the temperature given below for the month of June in North India. Calculate the average temperature and median value. This temperature gets dipped with a variation of 20°C in the month of December. Write a Python program to calculate the changed median value and average temperature.

Location	Temperature (in $^{\circ}\text{C}$)
Delhi	41
Shimla	32
Chandigarh	43
Rohtak	40
Srinagar	28
Sri Ganganagar	45

21. Create a menu-driven program using user-defined functions to implement a calculator that performs the following:
 (a) Basic arithmetic operations(+, -, *, /) (b) $\log_{10}(x)$, $\sin(x)$, $\cos(x)$
22. Write a program that contains user-defined functions to calculate area, perimeter or surface area, whichever is applicable, for various shapes like square, rectangle, triangle, circle and cylinder. The user-defined functions should accept the values for calculation as parameters and the calculated value should be returned. Import the module and use appropriate functions.

CASE-BASED/SOURCE-BASED INTEGRATED QUESTIONS

1. Road accidents occur due to various reasons. While problems with roads or safety facilities lead to some accidents, majority of the accidents are caused owing to drivers' not abiding by traffic rules.

MR Roadways Ltd. deals in the manufacture and installation of traffic lights so as to minimize the risk of accidents. Keeping in view the requirements, traffic simulation is to be done. Write a Python program that simulates a traffic light. The program should perform the following:

- (a) Develop a user-defined module/function `trafficLight()` that accepts input from the user, displays an error message if the user enters anything other than RED, YELLOW, and GREEN. Function `light()` is called and following is displayed depending upon return value from `light()`.
- "STOP, Life is more important than speed" if the value returned by `light()` is 0.
 - "Please GO SLOW, till the light is Green" if the value returned by `light()` is 1.
 - "You may GO NOW" if the value returned by `light()` is 2.
- (b) A user-defined function `light()` that accepts a string as input and returns 0 when the input is RED, 1 when the input is YELLOW and 2 when the input is GREEN. The input should be passed as an argument.
- (c) Display "BETTER LATE THAN NEVER" after the function `trafficLight()` is executed.

Ans.

```

prog_traffic_light.py - C:\Python382\prog_traffic_light.py (3.8.2)
File Edit Format Run Options Window Help
#Program to simulate a traffic light comprising of
#two user defined functions trafficLight() and
#light().
def trafficLight():
    signal = input("Enter the colour of the traffic light: ")
    if (signal not in ("RED", "YELLOW", "GREEN")):
        print("Please enter a valid Traffic Light colour in CAPITALS")
    else:
        value = light(signal) #function call to light()
        if (value == 0):
            print("STOP, Life is more important than speed")
        elif (value == 1):
            print("PLEASE GO SLOW.")
        else:
            print("You may go now.")

def light(colour):
    if (colour == "RED"):
        return(0)
    elif (colour == "YELLOW"):
        return(1)
    else:
        return(2)
#function ends here

trafficLight()
print("BETTER LATE THAN NEVER")

```

```

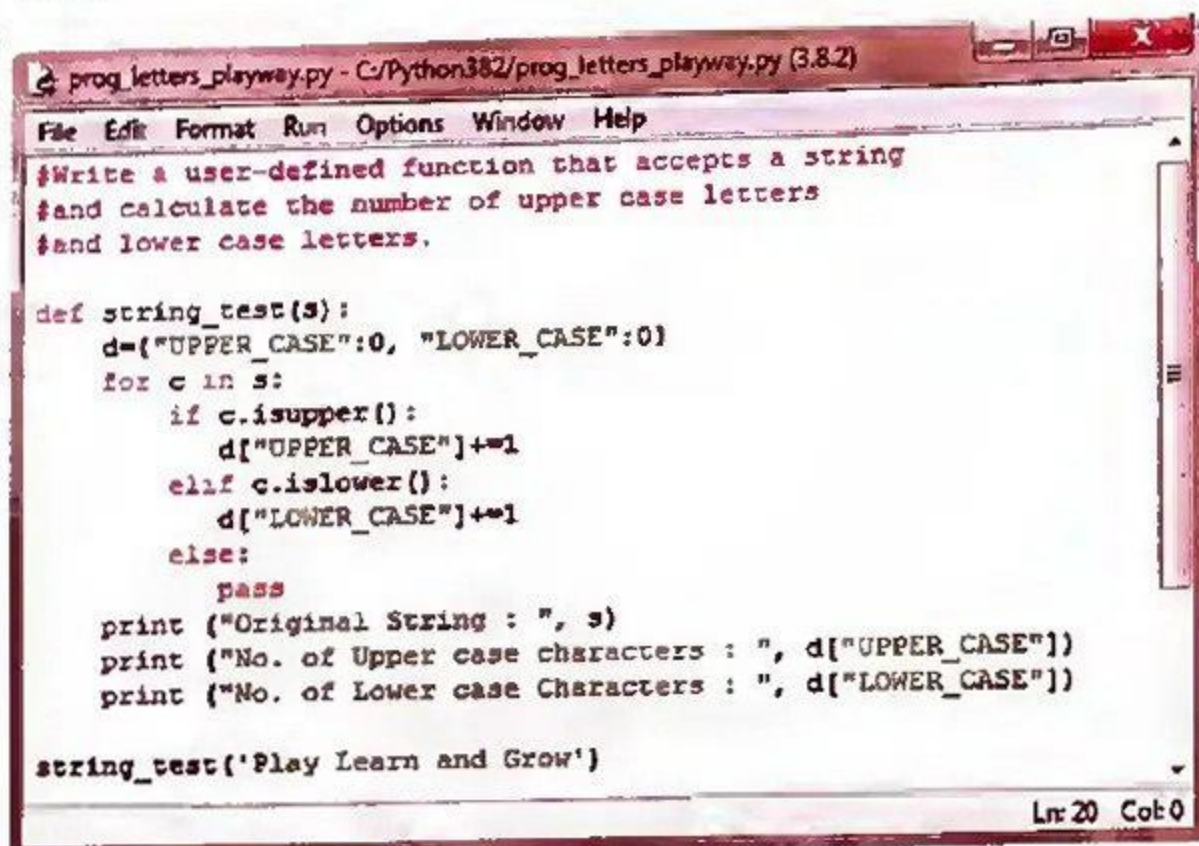
>>>
----- RESTART: C:/Python382/prog_traffic_light.py -----
Enter the colour of the traffic light: RED
STOP, Life is more important than speed
BETTER LATE THAN NEVER
>>>
----- RESTART: C:/Python382/prog_traffic_light.py -----
Enter the colour of the traffic light: YELLOW
PLEASE GO SLOW.
BETTER LATE THAN NEVER
>>>
----- RESTART: C:/Python382/prog_traffic_light.py -----
Enter the colour of the traffic light: GREEN
You may go now.
BETTER LATE THAN NEVER

```

2. Kids Elementary is a playway school that focuses on 'Play and learn' strategy that helps toddlers understand concepts in a fun way. Being a senior programmer, you have taken responsibility to develop a program using user-defined functions to help children differentiate between uppercase and lowercase letters/English alphabets in a given sentence. Make sure that you perform a careful analysis of the type of alphabets and sentences that can be included as per age and curriculum.

Write a Python program that accepts a string and calculates the number of uppercase and lowercase letters.

Ans.



```

File Edit Format Run Options Window Help
#Write a user-defined function that accepts a string
#and calculate the number of upper case letters
#and lower case letters.

def string_test(s):
    d={"UPPER_CASE":0, "LOWER_CASE":0}
    for c in s:
        if c.isupper():
            d["UPPER_CASE"]+=1
        elif c.islower():
            d["LOWER_CASE"]+=1
        else:
            pass
    print ("Original String : ", s)
    print ("No. of Upper case characters : ", d["UPPER_CASE"])
    print ("No. of Lower case Characters : ", d["LOWER_CASE"])

string_test('Play Learn and Grow')

Ln 20 Col 0

```

```

>>>
----- RESTART: C:/Python382/prog_letters_playway.py -----
Original String : Play Learn and Grow
No. of Upper case characters : 3
No. of Lower case Characters : 13

```